전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계



4주차 ROS2 프로그래밍 #2

1차시 ROS2 Topic

* 본 영상은 무단 배포할 수 없습니다.

학습내용

1 Workspace 개념

2 Topic 기반 ROS 구성 및 Topic 사용방법

학습목표

- Workspace 개념을 이해하고 설명할 수 있다.
- Topic 통신을 위한 Publisher/Subscriber를 만들어볼 수 있다.

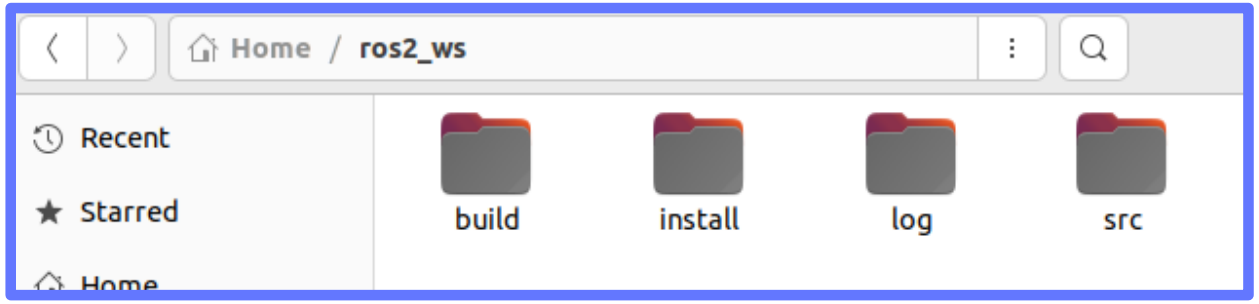
Workspace 개념

Workspace

Workspace 생성

패키지를 작성하기 위한 디렉토리

```
$ cd ~
$ mkdir -p ~/ros2_ws/src
$ cd ~/ros2_ws
$ colcon build --symlink-install
```



Workspace

Workspace 패키지 빌드

colcon

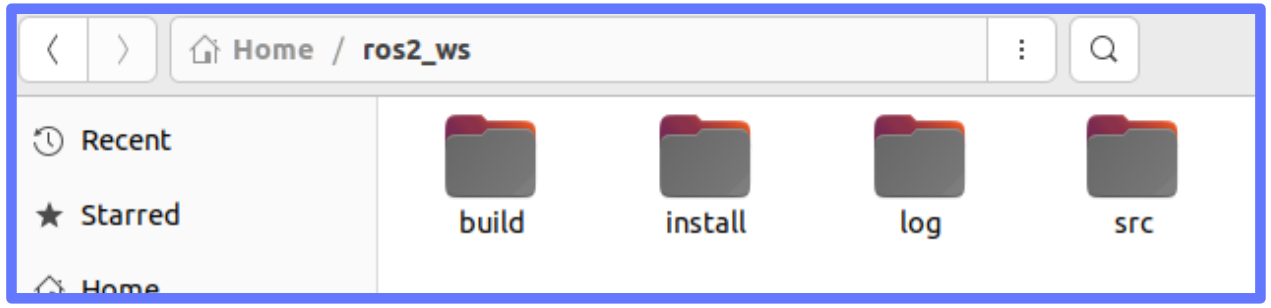
- 패키지를 빌드하기 위한 도구
- ROS 설치 시 자동 설치, 만약 설치 안 되어 있다면 아래 명령어 설치

```
$ sudo apt install python3-colcon-common-extensions
```

패키지 빌드

```
$ cd ~/ros2_ws
$ colcon build --symlink-install
```

- python, yaml 등의 파일 수정 후 다시 빌드하지 않기 위한 옵션



Workspace 환경 설정

.bashrc 파일 끝에 문구 추가

- 패키지를 환경 설정에 넣어 터미널 실행 시 자동으로 패키지를 인식하도록 하기 위함

```
source ~/ros2_ws/install/setup.bash
```

```
123 alias humble='source /opt/ros/humble/setup.bash; echo \ROS2 Humble!\ '
124 source ~/ros2_ws/insatll/setup.bash
125
```

추가 후 다시 설정 불러오기

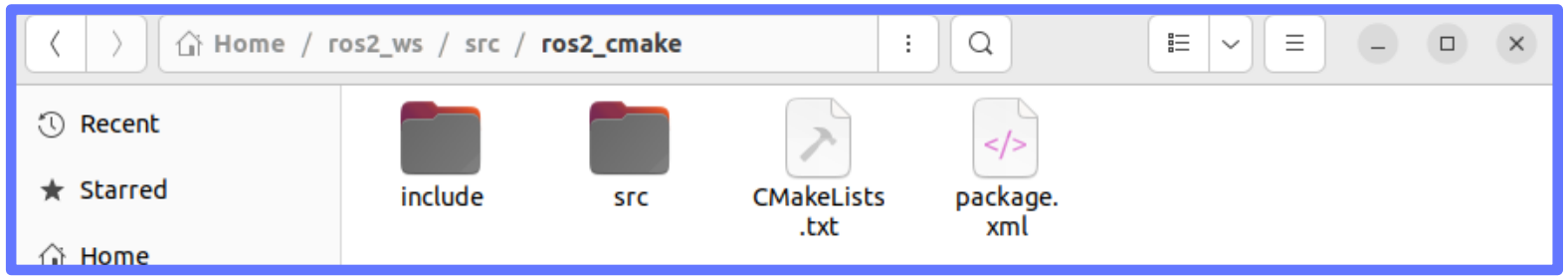
```
$ source ~/.bashrc
```

Topic 기반 ROS 구성 및 Topic 사용 방법

Package 생성 - cpp

C++ 사용하는 패키지 생성

```
$ cd ~/ros2_ws/src
$ ros2 pkg create --build-type ament_cmake ros2_cmake
```



Topic - cpp

publisher.hpp

```
$ cd ~/ros2_ws/src/ros2_cmake/include/ros2_cmake/
$ code publisher.hpp
```

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
```

관련 header 파일 include

Topic - cpp

publisher.hpp

```
using namespace std::chrono_literals;

class StringPublisher : public rclcpp::Node
{
 public:
   StringPublisher(std::string node_name, std::string topic_name)
   : Node(node_name) {
       publisher_ = this->create_publisher<std_msgs::msg::String>(topic_name, 10);
       timer_ = this->create_wall_timer(500ms, std::bind(&StringPublisher::timer_callback, this));
   }
```

ROS Node로 Class 정의

Publisher 생성

Publish 함수

Topic - cpp

publisher.hpp

```
private:
  void timer_callback()
  {
    auto message = std_msgs::msg::String();
    message.data = "Hello, world! " + std::to_string(count_++);
    RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
    publisher_->publish(message);
  }
  rclcpp::TimerBase::SharedPtr timer_;
  rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
  size_t count_;
};
```

Message 정의

Node가 해당 Message를 Publish

Topic - cpp

publisher.cpp

```cpp
#include "ros2_cmake/publisher.hpp"

int main(int argc, char * argv[]) {
    std::string node_name = "ex_pub_node";
    std::string topic_name = "/ex_message";
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<StringPublisher>(node_name, topic_name));
    rclcpp::shutdown();
    return 0;
}
```

해당 노드 계속 실행

Topic - cpp

subscriber.hpp

```
$ cd ~/ros2_ws/src/ros2_cmake/include/ros2_cmake/
$ code subscriber.hpp
```

```cpp
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
using std::placeholders::_1;
class StringSubscriber : public rclcpp::Node
{
 public:
   StringSubscriber(std::string node_name, std::string topic_name)
   : Node(node_name) {
     subscription_ = this->create_subscription<std_msgs::msg::String>(
     topic_name, 10, std::bind(&StringSubscriber::topic_callback, this, _1));
   }
```

Subscriber 생성

Subscriber가 Message를 받을 경우 실행되는 함수

Topic - cpp

subscriber.hpp

```
private:
  void topic_callback(const std_msgs::msg::String & msg) const
  {
    RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
  }
  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};
```

Subscriber가 Message를 받을 경우 실행되는 함수

Message 출력

Topic - cpp

subscriber.cpp

```cpp
#include "ros2_cmake/subscriber.hpp"

int main(int argc, char * argv[]) {
    std::string node_name = "ex_sub_node";
    std::string topic_name = "/ex_message";
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<StringSubscriber>(node_name, topic_name));
    rclcpp::shutdown();
    return 0;
}
```

Topic - cpp

CMakeLists.txt

```
# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
include_directories(include)
# topic - publisher
add_executable(publisher src/publisher.cpp)
ament_target_dependencies(publisher rclcpp std_msgs)
# topic - subscriber
add_executable(subscriber src/subscriber.cpp)
ament_target_dependencies(subscriber rclcpp std_msgs)
install(TARGETS
 # topic
 publisher
 subscriber
 DESTINATION lib/${PROJECT_NAME})
```

관련 패키지 찾기

include 디렉토리 추가

실행 파일

의존성 추가

실행 파일 이름

Topic - cpp

package.xml

```
...
<buildtool_depend>ament_cmake</buildtool_depend>

<depend>rclcpp</depend>
<depend>std_msgs</depend>
...
```

관련 의존성 추가

Topic - cpp

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_cmake
```

빌드 후 패키지 환경 다시 불러오기

```
$ source install/setup.bash
```

Topic - cpp

실행

터미널 1
$ ros2 run ros2_cmake publisher

터미널 2
$ ros2 run ros2_cmake subscriber

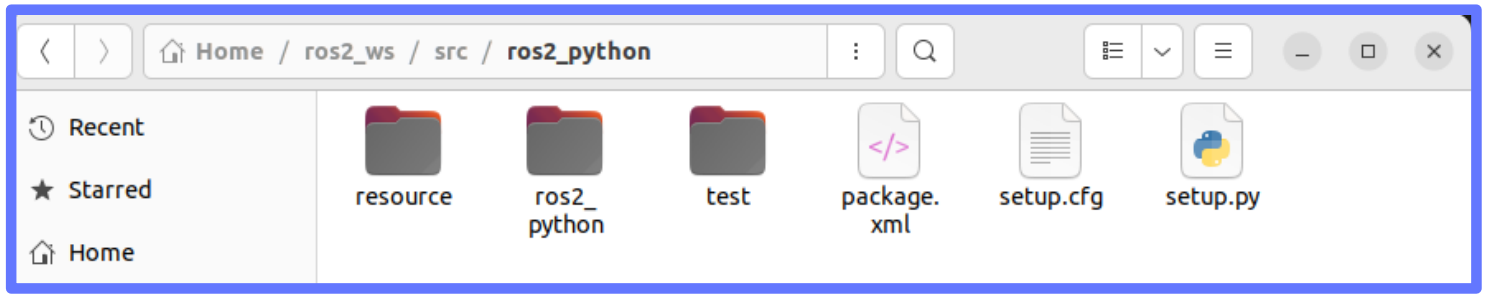
터미널 1 출력
[INFO] [1735278891.364182823] [ex_pub_node]: Publishing: 'Hello, world! 0' [INFO] [1735278891.864424660] [ex_pub_node]: Publishing: 'Hello, world! 1'

터미널 2 출력
[INFO] [1735278891.364681075] [ex_sub_node]: I heard: 'Hello, world! 0' [INFO] [1735278891.864959998] [ex_sub_node]: I heard: 'Hello, world! 1'

Topic - python

패키지 생성

```
$ cd ~/ros2_ws/src
$ ros2 pkg create --build-type ament_python ros2_python
```



Topic - python

publisher.py

```
$ cd ~/ros2_ws/src/ros2_python/ros2_python/
$ code publisher.py
```

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
```

필요한 라이브러리 Import

Topic - python

publisher.py

```
class StringPublisher(Node):                                    Class 정의

    def __init__(self, node_name, topic_name):
        super().__init__(node_name)                             Publisher 생성
        self.publisher_ = self.create_publisher(String, topic_name, 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)                            Message를 Publish
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1
```

Topic - python

publisher.py

```
def main(args=None):
    rclpy.init(args=args)
    node_name = "ex_pub_node_py"
    topic_name = "/ex_message_py"
    string_publisher = StringPublisher(node_name, topic_name) Node 생성

    rclpy.spin(string_publisher) Node 실행
    string_publisher.destroy_node() Node 종료
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

Topic - python

subscriber.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
class StringSubscriber(Node):
    def __init__(self, node_name, topic_name):
        super().__init__(node_name)
        self.subscription = self.create_subscription(
            String,
            topic_name,
            self.listener_callback,
            10)
        self.subscription

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)
```

Subscriber 생성

Subscriber가 Message를 받을 경우 실행되는 함수

Topic - python

subscriber.py

```
def main(args=None):
    rclpy.init(args=args)
    node_name = "ex_sub_node_py"
    topic_name = "/ex_message_py"
    string_subscriber = StringSubscriber(node_name, topic_name)
    rclpy.spin(string_subscriber)
    string_subscriber.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

Topic - python

package.xml

```
...
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
...
```

관련 의존성 추가

Topic - python

setup.py

```
entry_points={
        'console_scripts': [
            'ex_pub_node_py = ros2_python.publisher:main',
            'ex_sub_node_py = ros2_python.subscriber:main',
        ],
    },
```

실행 파일 작성

'Node 이름(실행 파일) = 패키지이름.파이썬 파일 이름:main'

Topic - python

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_python
```

빌드 후 다시 불러오기

```
$ source install/setup.bash
```

Topic - python

실행

터미널 1	터미널 2
$ ros2 run ros2_python ex_pub_node_py	$ ros2 run ros2_python ex_sub_node_py

터미널 1 출력	터미널 2 출력
[INFO] [1735279308.219978883] [ex_pub_node_py]: Publishing: "Hello World: 0" [INFO] [1735279308.707299011] [ex_pub_node_py]: Publishing: "Hello World: 1"	[INFO] [1735279308.220422018] [ex_sub_node_py]: I heard: "Hello World: 0" [INFO] [1735279308.707636898] [ex_sub_node_py]: I heard: "Hello World: 1"